System Verification and Validation Plan for Keystone

Team 13

Ayush Patel
Ahmed Rashrash
Andrew Habib
Waqar Ul-Hassan

November 1, 2026

Revision History

Date	Version	Notes
2026-10-29	1.0	General Information

Contents

1 Symbols, Abbreviations, and Acronyms **iv**

2 General Information **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 1
- 2.3 Custom Documents . . . 3
- 2.4 Relevant Documentation . . . 4

3 Plan **5**
- 3.1 Verification and Validation Team . . . 5
- 3.2 SRS Verification . . . 5
- 3.3 Design Verification . . . 5
- 3.4 Verification and Validation Plan Verification . . . 5
- 3.5 Implementation Verification . . . 5
- 3.6 Automated Testing and Verification Tools . . . 5
- 3.7 Software Validation . . . 5

4 System Tests **5**
- 4.1 Tests for Functional Requirements . . . 5
  - 4.1.1 Area of Testing1 . . . 5
  - 4.1.2 Area of Testing2 . . . 6
- 4.2 Tests for Nonfunctional Requirements . . . 6
  - 4.2.1 Usability . . . 6
  - 4.2.2 Installability . . . 6
  - 4.2.3 Portability . . . 7
- 4.3 Traceability Between Test Cases and Requirements . . . 7

5 Unit Test Description **7**
- 5.1 Unit Testing Scope . . . 7
- 5.2 Tests for Functional Requirements . . . 7
  - 5.2.1 Module 1 . . . 7
  - 5.2.2 Module 2 . . . 8
- 5.3 Tests for Nonfunctional Requirements . . . 8
  - 5.3.1 Module ? . . . 8
  - 5.3.2 Module ? . . . 8
- 5.4 Traceability Between Test Cases and Modules . . . 8

6 Appendix **9**
6.1 Symbolic Parameters 9
6.2 Usability Survey Questions? 9

7 Appendix — Generative AI Use Disclosure **10**
7.1 AI Tools Used 10
7.2 How and Where Used 10
7.3 Verification of AI-Generated Content 10

8 Appendix — Reflection **11**

List of Tables

List of Figures

1 Symbols, Abbreviations, and Acronyms

symbol	description
T	Test

This document ...

2 General Information

2.1 Summary

The software under test is Keystone, a web-based capstone project management system for the Software Engineering (SE), Computer Science (CS), and Mechatronics (TRON) capstone courses. Keystone covers the lifecycle from project proposal to team project approval. Its main functions are:

- structured project proposals by supervisors and student teams, with instructor review before a project is listed;
- a searchable project catalog showing project status and supervisor capacity;
- student and team profiles, team formation under course-specific rules, and teammate and project recommendations;
- team applications to projects, supervisor and team acceptance, and comment threads attached to projects and applications;
- team project submission with automated checks, and approval by instructors and TAs, either in the platform or recorded from an external process;
- course enrollment import from Avenue to Learn or file upload, validated team composition export, and email notifications and checklists;
- role-based access control, authentication through McMaster SSO or another trusted provider, and audit logging.

The requirements being verified are listed in the SRS (Author, 2019).

2.2 Objectives

The objectives are ordered by priority, based on the risks identified in the SRS: errors in the approval workflow affect course records, errors in access

control expose student data, and a hard-to-use platform will not be adopted by supervisors.

In scope.

1. **Functional correctness.** Build confidence that the functional requirements (`FR_SupervisorProposal` to `FR_AssignTA` in the SRS) are met, especially the project and team-project status lifecycle, supervisor capacity (slot reservation and release), team formation rules, submission validation, enrollment import, and external supervisor verification and scoping.

2. **Access control and privacy.** Demonstrate that users can only see and change what their role and course permit, and that data is protected in transit and at rest (`NFR_Authenticate`, `NFR_Authorize`, `NFR_EncryptTransit`, `NFR_EncryptRest`, `DPR_RoleBasedDataPermissions`, `DPR_VisibilityMatrix`, `DPR_EncryptionAtRest`).

3. **Usability.** Demonstrate that first-time users in the main roles (students, TAs, and professors) can complete their primary tasks within the usability time limit, and find the platform intuitive and easy to use (`NFR_UsabilityFirstSuccess`, `NFR_LearnUnassisted`).

4. **Performance and installability.** Demonstrate that the system meets its latency target under the specified load, and that an IT professional can deploy it from the deployment guide within the time limit (`NFR_PerfTail`, `NFR_PeakUsers`, `NFR_DeployUnassisted`).

5. **Portability and accessibility.** Demonstrate that the system works on the target browsers and in Docker, and passes an automated accessibility audit (`NFR_Browsers`, `NFR_Container`, `NFR_AccessibilityWCAG`).

Out of scope.

- **Quality of recommendations.** We will test that recommendations are produced and displayed (`FR_Label12`), but not whether they are good matches. Recommendation quality is a stretch goal in the SRS, and judging it would need a study with real students.

- **External systems.** We assume that McMaster SSO, Avenue to Learn, the email service, and third-party libraries have been verified by their

developers. We test our handling of their failures (for example, an unavailable Avenue), not their correct operation. Following the SRS Development Plan, email delivery and authentication may be simulated during the course.

- **Availability over time.** The availability criterion is measured over a long window (see the SRS), which cannot be observed within the course timeline. We will verify that the health check works and that the system meets the target over the period that the demo allows.
- **Maintainability.** We will review the likely changes in the SRS against the design, but will not measure the actual effort of making changes.
- **Features not implemented.** Requirements that the SRS Development Plan marks as future work, or as Phase 3 and not completed, will not be tested.
- **Formal verification.** We do not plan formal proofs.
- **Industry sponsors.** The usability test does not include external supervisors from industry, who have no McMaster account and are the least familiar with the course. Professors in the test represent internal supervisors and instructors. Findings about external supervisors will need follow-up testing if one becomes available.

2.3 Custom Documents

This project has two Custom Documents: a usability testing report and an interview report.

The **usability testing report** records the results of the usability test described in Section 4.2.1 and supports the Usability NFR in the SRS. In the test, we observe TAs, students, and professors using Keystone to complete the primary tasks of their roles, to judge whether the platform is intuitive and easy to use without training. The report covers the participants and their roles, the tasks given, completion times, problems and points of confusion observed, and the changes made as a result.

The **interview report** records the stakeholder interviews that the requirements were derived from: students, external and internal supervisors, and the SE, TRON, and CS course instructors. It contains the interview

notes, the requirements and design decisions each note led to, and the stakeholder feedback on the SRS and on the working system. It supports validation by checking that Keystone solves the problems stakeholders described, and the SRS (Appendix A) summarizes it.

2.4 Relevant Documentation

The following documents define what is verified and how the verification is organized.

- **Software Requirements Specification (SRS)** (Author, 2019). This is the main source for this plan. System tests are derived from its functional requirements, nonfunctional requirements and fit criteria, use cases, and data requirements. The SRS defines the symbolic constants (such as the number of concurrent users and the latency limit) that the nonfunctional tests use, and its traceability matrices will be extended to link each requirement to the tests in this document.

- **Hazard Analysis** (**?**). It identifies hazards such as failures of the external approval system and access control failures, and the safety and security requirements that follow from them. Tests are added for each of these requirements.

- **Problem Statement and Goals** (**?**). It records the problems and goals that validation checks against. Validation with stakeholders asks whether the system solves these problems, not only whether it meets the SRS.

- **Development Plan** (**?**). It defines the tools and workflow (such as version control, continuous integration, and linting) that the automated testing in this plan relies on.

- **Module Guide (MG) and Module Interface Specification (MIS)** (**??**). These will define the modules and interfaces that the unit tests in this plan target. The unit test sections will be completed after the design is finished.

3 Plan

3.1 Verification and Validation Team

3.2 SRS Verification

3.3 Design Verification

3.4 Verification and Validation Plan Verification

3.5 Implementation Verification

3.6 Automated Testing and Verification Tools

3.7 Software Validation

4 System Tests

4.1 Tests for Functional Requirements

4.1.1 Area of Testing1

Title for Test

1. test-id1

   Control: Manual versus Automatic
   Initial State:
   Input:
   Output:
   Test Case Derivation:
   How test will be performed:

2. test-id2

   Control: Manual versus Automatic
   Initial State:

Input:

Output:

Test Case Derivation:

How test will be performed:

4.1.2 Area of Testing2

...

4.2 Tests for Nonfunctional Requirements

4.2.1 Usability

..to be completed.

Title for Test

1. test-id1

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input/Condition:

   Output/Result:

   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

   How test will be performed:

4.2.2 Installability

..to be completed.

4.2.3 Portability

..to be completed.

...

4.3 Traceability Between Test Cases and Requirements

5 Unit Test Description

5.1 Unit Testing Scope

5.2 Tests for Functional Requirements

5.2.1 Module 1

1. test-id1

   Type:
   Initial State:
   Input:
   Output:
   Test Case Derivation:
   How test will be performed:

2. test-id2

   Type:
   Initial State:
   Input:
   Output:
   Test Case Derivation:
   How test will be performed:

3. ...

5.2.2 Module 2

...

5.3 Tests for Nonfunctional Requirements

5.3.1 Module ?

1. test-id1

   Type:

   Initial State:

   Input/Condition:

   Output/Result:

   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

   How test will be performed:

5.3.2 Module ?

...

5.4 Traceability Between Test Cases and Modules

References

Author Author. System requirements specification. https://github.com/..., 2019.

6 Appendix

This is where you can place additional information.

6.1 Symbolic Parameters

The definition of the test cases will call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance.

6.2 Usability Survey Questions?

7 Appendix — Generative AI Use Disclosure

7.1 AI Tools Used

7.2 How and Where Used

7.3 Verification of AI-Generated Content

8 Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

Ayush:

1. What went well while writing this deliverable?

   I think it was pretty easy to come up with this plan. We all have extensive experience with verification and validation from our course work and industry experience: unit testing, integration testing, E2E testing, and CI/CD and automation pipelines. We also know how important it is to validate with customers that we are building the right thing, and I have experience with regular demoing to a customer for exactly that reason. All of our past experience has allowed us to come up with a good plan for this deliverable.

2. What pain points did you experience during this deliverable, and how did you resolve them?

   Time management and prioritization with so much going on right now. The plan itself came together easily, but finding the time to actually sit down and write it up was hard with midterms and other assignments all landing around the same time. I resolved this by blocking out a few set times in the week for this deliverable and writing it in smaller chunks, instead of waiting for a big free afternoon that never showed up.

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

   For me, it is unit testing in Python with pytest and in React with Jest. I have experience with Vitest in Angular, but I have never unit tested a React project, so I expect the ideas to carry over but the tooling to feel new. As a team, we will also need Playwright for the end-to-end tests, integration and concurrency testing against a real PostgreSQL database (including stubbing Avenue and the email service), and setting up Locust load tests and mutation testing in GitHub Actions.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

   For React unit testing, two approaches are reading the Jest and React Testing Library documentation, and building a small throwaway React app with a few component tests to get hands-on practice. I will do both: the documentation first for the basics, then the small project, because I learn fastest by doing things. YouTube tutorials are a good backup when the documentation does not make something click. The same two approaches (documentation plus a small practice project) work for the other skills on the team list, and the rest of the team will pick whichever suits them for the tool they own.